

AvgPool1d#

<https://pytorch.org/docs/stable/generated/torch.nn.AvgPool1d.html#torch.nn.AvgPool1d>

Description:

Applies a 1D average pooling over an input signal composed of several input planes. In the simplest case, the output value of the layer with input size (N, C, L) , output (N, C, L_{out}) and kernel_size k can be precisely described as: If padding is non-zero, then the input is implicitly zero-padded on both sides for padding number of points. When `ceil_mode=True`, sliding windows are allowed to go off-bounds if they start within the left padding or the input. Sliding windows that would start in the right padded region are ignored.

Function Signature

```
class torch.nn.AvgPool1d(kernel_size, stride=None,  
padding=0, ceil_mode=False, count_include_pad=True)[source]#
```

Parameters

Shape:

```
forward(input)[source]#
```

Return type

Returns:

Tensor

Code Examples

```
>>> # pool with window of size=3, stride=2  
>>> m = nn.AvgPool1d(3, stride=2)  
>>> m(torch.tensor([[[1., 2, 3, 4, 5, 6, 7]]]))  
tensor([[[2., 4., 6.]])
```

Notes & Warnings

NOTE

Note When `ceil_mode=True`, sliding windows are allowed to go off-bounds if they start within the left padding or the input. Sliding windows that would start in the right padded region are ignored.

NOTE

Note `pad` should be at most half of effective kernel size.

Detailed Documentation

Documentation

Applies a 1D average pooling over an input signal composed of several input planes.

In the simplest case, the output value of the layer with input size (N, C, L) , output (N, C, L_{out}) and kernel_size kkk can be precisely described as:

If padding is non-zero, then the input is implicitly zero-padded on both sides for padding number of points.

When `ceil_mode=True`, sliding windows are allowed to go off-bounds if they start within the left padding or the input. Sliding windows that would start in the right padded region are ignored.

`pad` should be at most half of effective kernel size.

The parameters `kernel_size`, `stride`, `padding` can each be an int or a one-element tuple.

`kernel_size` (`Union[int, tuple[int]]`) – the size of the window

`stride` (`Union[int, tuple[int]]`) – the stride of the window. Default value is `kernel_size`

`padding` (`Union[int, tuple[int]]`) – implicit zero padding to be added on both sides

`ceil_mode` (bool) – when True, will use ceil instead of floor to compute the output shape

`count_include_pad` (bool) – when True, will include the zero-padding in the averaging calculation

Input: (N, C, L_{in}) or (N, C, L_{in}) or (C, L_{in}) or (C, L_{in})

Output: (N, C, L_{out}) or (N, C, L_{out}) or (C, L_{out}) or (C, L_{out}) , where

Per the note above, if `ceil_mode` is True and $(L_{out}-1) \times \text{stride} \geq L_{in} + \text{padding}$, we skip the last window as it would start in the right padded region, resulting in L_{out} being reduced by one.

Runs the forward pass.